Q1 Power function , a & n find a^n

⇒

```
int power (int a, int n) {
    // base
    if (n == 0) return 1;

    // normal case
    return a * power (a, n-1);
}
```

Optimisation

$2^6 = \underbrace{2 \times 2 \times 2}_{} \times \underbrace{2 \times 2 \times 2}_{}$

can we like break down the power?

$2^6 = 2^3 \times 2^3$

$2 \times 2^1 \quad 2^1$

2^9

$2 \times 2^{9/2} \quad 2^{9/2}$

$2^9 \Rightarrow$ odd

$a \times$

$2 \times 2^4 \times 2^4$

$2^2 \times 2^2 \quad 2^2 \quad 2^2$

$2 \times 2 \quad 2 \times 2 \quad 2 \times 2 \quad 2 \times 2$

⇒ if power is odd a^n = a × power(a, n/2) × power(a, n/2)

is even a^n = power(a, n/2) × power(a, n/2)

```
long powerC int a, int n) {

    // edge cases
    if (a == 0) return 0;
    if (a == 1) return 1;

    // base case
    if (n == 0) return 1;

    if (n % 2 == 0) {
        return pow(a, n/2) * pow(a, n/2)
    } else {
        return a * pow(a, n/2) * pow(a, n/2);
    }
```

a^2 = return pow(a,1) × pow(a, 1)

a × a

return a × pow(a, 0) × pow(a, 0)

a^3 = return a × pow(a,2) × pow(a, 1)

a × a × a

ret a × pow(a, 0) × pow(a, 0)

⇒ optimize further

```
long halfPower = power(a, n/2);

if (n % 2 == 0) {
    return halfpower * halfpower;
} else {
    return (long) a * halfpower * halfpower;
}
```

DRY RUN

2^5

```
power (2, 5) {  = 32
    halfPower = power (2, 2) {  = 4
        halfPower = power (2, 1) {  = 2
            halfPower = power (2, 0) {  = 1
                return 1;
            }
            1 % 2 != 0
            return 2 * 1 * 1;
        }
        2 % 2 == 0
        return 2 * 2 = 4
    }
    5 % 2 != 0
    return 2 * 4 * 4  = 32
}
```